

EDUOS HYBRID OPERATING SYSTEM SIMULATOR

Name : **Precious Mpita**
Course : **Operating Systems**
Department : **CS & IT Department**
Date : **24 May 2026**

INTRODUCTION

The EduOS Hybrid Operating System Simulator is a project developed using both Python and C programming languages to simulate important operating system concepts. The system demonstrates process scheduling, process management, threading, synchronization, inter-process communication (IPC), and system visualization.

Python was mainly used for implementing CPU scheduling algorithms, generating metrics, visualizing Gantt charts, and controlling system execution. C was used for low-level operating system concepts such as thread management, mutex synchronization, race condition demonstrations, and IPC implementation.

The project was designed to provide practical understanding of how modern operating systems manage processes and resources.

OBJECTIVES

The main objectives of this project were:

- 1.To implement CPU scheduling algorithms
- 2.To simulate process control block (PCB) management
- 3.To demonstrate thread creation and synchronization
- 4.To implement IPC communication using pipes
- 5.To visualize scheduling performance using Gantt charts
- 6.To compare scheduling algorithm performance
- 7.To integrate Python and C modules into one hybrid system

TECHNOLOGIES USED

TECHNOLOGY	PURPOSE
Python	Scheduling algorithms and controller
C	Low-level OS implementation
GCC	C compilation
Matplotlib	Graph generation
Pthread	Thread management
Git/GitHub	Version control
CSV	Process input handling

SYSTEM ARCHITECTURE

The project follows a hybrid architecture combining Python and C modules.

#Python components handle:

- i). Scheduling algorithms
- ii). Metrics calculation
- iii). Visualization
- iv). Main controller

#C components handle:

- i). Process management
- ii). Thread management
- iii). IPC communication
- iv). Mutex synchronization

*The Python controller communicates with the C core to demonstrate integration between high-level and low-level programming.

PROCESS CONTROL BLOCK (PCB)

The Process Control Block (PCB) stores information about each process in the system.

Each PCB contains:

- i). Process ID (PID)**
- ii). Arrival Time**
- iii). Burst Time**
- iv). Priority**
- v). Process State**

*The PCB structure allows the scheduler to track process execution and calculate scheduling metrics such as waiting time and turnaround time.

CPU SCHEDULING ALGORITHMS

I). First Come First Serve (FCFS)

FCFS schedules processes according to their arrival time. The first arriving process executes first until completion.

Advantages:

- Simple to implement
- Fair in arrival order

Disadvantages:

- Can produce long waiting times
- Suffers from convoy effect

II). Shortest Job First (SJF)

SJF selects the process with the shortest burst time.

Advantages:

- Produces lower average waiting time
- Efficient for short processes

Disadvantages:

- Can cause starvation for longer jobs

III). Priority Scheduling

Processes are executed according to priority values.

Advantages:

- Important tasks execute first

Disadvantages:

- Low-priority processes may starve

IV). Round Robin (RR)

Round Robin assigns a fixed time quantum to each process.

Advantages:

- Fair CPU sharing
- Good for time-sharing systems

Disadvantages:

-Performance depends on quantum size

SCHEDULING RESULTS AND ANALYSIS

The scheduling algorithms were tested using sample process datasets loaded from CSV files.

The simulator generated:

1. Waiting Time
2. Turnaround Time
3. Response Time
4. CPU Utilization
5. Throughput

Gantt charts and comparison charts were produced to visualize scheduling performance.

In many tests, SJF produced lower average waiting times compared to FCFS and Priority Scheduling.

THREADING AND SYNCHRONIZATION

The project implemented multithreading using pthreads in C.

Two demonstrations were created:

-race_demo.c
-fixed_demo.c

*The race_demo.c program demonstrated a race condition where multiple threads updated a shared counter simultaneously without synchronization. This produced inconsistent and incorrect results.

*The fixed_demo.c program used mutex locks to protect shared resources. The mutex ensured that only one thread accessed the counter at a time, producing correct results consistently.

This demonstrated the importance of synchronization in concurrent systems.

INTER-PROCESS COMMUNICATION (IPC)

IPC communication was implemented using pipes in C.

The IPC module demonstrated:

1. Message passing between processes
2. Communication using file descriptors
3. Basic process synchronization

The IPC implementation showed how operating system processes exchange data safely.

C VS PYTHON LESSONS LEARNED

This project demonstrated the differences between Python and C programming in operating system development.

Python provided:

- Faster development
- Easier debugging
- Simpler visualization
- Cleaner syntax

C provided:

- Lower-level hardware interaction
- Better understanding of memory management
- Direct thread management
- Greater control over synchronization and IPC

Python was more suitable for scheduling simulations and data visualization, while C was more suitable for implementing low-level operating system concepts.

Using both languages together demonstrated the advantages of hybrid system design.

CHALLENGES FACED AND SOLUTIONS

Challenge 1: GCC Configuration

- The GCC compiler was initially not recognized in PowerShell.

Solution

- The MinGW bin directory was added to the Windows PATH environment variable.

Challenge 2: CSV Parsing Problems

-The scheduler initially failed to load processes from the CSV file correctly.

Solution

-The csv.DictReader module was used and file formatting issues were corrected.

Challenge 3: Thread Synchronization

-The shared counter produced inconsistent values due to race conditions.

Solution

-Mutex locks were implemented using pthread_mutex_lock() and pthread_mutex_unlock().

Challenge 4: Module Import Errors

Python modules were initially not detected correctly.

Solution

-The project structure and import paths were reorganized properly.

SYLLABUS MAPPING TABLE

Syllabus Topic	EduOS Implementation	File
CPU Scheduling	FCFS, SJF, RR, Priority	schedulers.py
PCB Management	Process structures	pcb.py
Threading	pthread threads	thread_manager.c
Synchronization	Mutex locks	fixed_demo.c
IPC	Pipes	ipc_module.c
Visualization	Gantt Charts	gantt.py

GITHUB SUMMARY

The project was managed using Git and GitHub for version control.

GitHub Features Used:

- Repository management
- Commit tracking
- Project backup
- Version history

*The repository contains:

- Source code
- Documentation
- Screenshots
- Report
- Charts
- README

Meaningful commits were made throughout development to track project progress.

CONCLUSION

The EduOS Hybrid Operating System Simulator successfully demonstrated major operating system concepts including process scheduling, threading, synchronization, IPC, and hybrid Python/C integration.

The project provided practical understanding of how operating systems manage processes and system resources. It also demonstrated the advantages of combining high-level and low-level programming languages in systems development.

The implementation of visualization tools, race condition demonstrations, and scheduling comparisons made the simulator educational and interactive.

REFERENCES

- Python Documentation
- GCC Documentation
- pthread Documentation
- Operating System Concepts by Silberschatz
- matplotlib Documentation
- GitHub Documentation